

Employee Records Manager

Persistent Flat-File Subsystems & CRUD Architecture

TRAINEE: Benatic Rithan B

INSTITUTION: KIT, Coimbatore

DATE: 12 June 2026

MENTOR: Rathees G

DEPARTMENT: CSE

STATUS: Completed Successfully

1. Executive Summary

This technical report structures the development of a persistent, file-based **Employee Records Manager** built during Day 5 of the internship pipeline. The application marks a transition from volatile in-memory collections to a persistent data tier by combining structured tabular models (CSV), configuration hierarchies (JSON), and sequential tracking files (TXT). The system implements a complete Create, Read, Update, and Delete (CRUD) subsystem optimized for execution integrity and automated data flushes.

2. Data Subsystem Mechanics

The persistence tier abstracts three discrete file execution methodologies to maximize transaction isolation:

- **Tabular Engine (CSV):** Implements low-overhead column matrices to store employee entity attributes including unique keys, names, roles, and salaries.
- **Structural Configurations (JSON):** Manages metadata parameters, target path variables, and schema boundaries to ensure configuration elasticity.
- **Transaction Stream (TXT):** Captures state updates and tracking vectors in an append-only transaction stream to provide audit trails.

Resource Protection Principle: System transactions systematically encapsulate system IO processes within Python context managers (with blocks) to ensure immediate system file descriptor releases at the OS kernel boundary.

3. CRUD Execution Matrix

Operation	Underlying Physical File Mechanic	Disk Synchronization Vector
Create	Appends a validation-checked discrete text array line directly to the data file.	Immediate Append Flushes
Read	Streams tabular disk contents into a high-speed runtime dictionary array.	Shared Read Locks
Update	Modifies the target profile attributes inside an isolated application memory buffer.	Atomic File Rewrite Overwrites
Delete	Filters out selected row arrays before committing remaining entries back down to disk.	Atomic File Rewrite Overwrites

4. Core Technical Architecture

The code segment below showcases the structure of the data layer engine:

```
import csv
import os
from datetime import datetime

class EmployeeManager:
    def __init__(self, csv_path="employees.csv", log_path="activity_log.txt"):
        self.csv_path = csv_path
        self.log_path = log_path
        self.headers = ["emp_id", "name", "department", "salary"]
        if not os.path.exists(self.csv_path):
            with open(self.csv_path, mode='w', newline='', encoding='utf-8') as f:
                csv.writer(f).writerow(self.headers)

    def _log(self, action, status):
        t = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        with open(self.log_path, mode='a', encoding='utf-8') as f:
            f.write(f"[{t}] [{action}] {status}\n")

    def create(self, emp_id, name, dept, salary):
        with open(self.csv_path, mode='a', newline='', encoding='utf-8') as f:
            csv.writer(f).writerow([emp_id, name, dept, float(salary)])
        self._log("CREATE", f"Successfully registered employee ID: {emp_id}")
```

5. Storage Ingestion Mockups

Tabular Workspace File (employees.csv)

```
emp_id,name,department,salary
E101,Benatic Rithan,Engineering,95000.0
E102,Rathees G,Research,105000.0
```

Transaction Stream Tracking (activity_log.txt)

```
[2026-06-12 11:20:04] [SYSTEM] Persistent storage targets initialized.
[2026-06-12 11:22:15] [CREATE] Successfully registered employee ID: E101
[2026-06-12 11:25:31] [UPDATE] Transformed attribute state for employee target ID: E102
```

6. Learning Outcomes

- **File Operations & Persistence:** Mastered the application of distinct flat-file paradigms (CSV, JSON, TXT) to manage structured, unstructured, and hierarchical data models.
- **CRUD Lifecycle Implementation:** Formulated structured logic for mutating persistent local structures while maintaining absolute structural alignment across text channels.
- **Resource Lifecycle Management:** Gained hands-on experience using explicit runtime context boundaries (with wrappers) to guarantee immediate lock clearance at the kernel boundary.
- **System Audit Logging:** Constructed append-only file tracking modules to provide full observability and diagnostic traceability for data engineering workflows.